

Chess Tournament Entry Prediction App — Project Brief

Overview

A SaaS dashboard for chess tournament directors that ingests live registration data, applies predictive modeling, and delivers actionable alerts when entries deviate from expectations. The initial use case is the **World Open** (annual, Philadelphia, ~400–800 entries), expandable to any CCA-run or USCF-listed tournament.

Data Sources

1. Historical Data (Training Set)

- **5 years** of past tournament registration data
- Fields needed per entry: `player_name`, `registration_timestamp`, `tournament_id`, `section`
- Tournament metadata: prize fund, entry fee tiers + deadlines, location, date, sections offered, confirmed titled players (GMs/IMs)

2. Live Registration Data (Web Scraping)

- Primary target: `chessaction.com` entry list pages (HTML tables)
- Also: `onlineregistration.cc`, possibly `uschess.org` event listings
- Login credentials provided by tournament director for authenticated pages
- Scraping frequency: **every 6 hours** (adjustable)
- Fields to extract: player name, registration timestamp, section (if visible)

3. Tournament Metadata (Manual + Scraped)

- Entry fee tiers and deadline dates (early bird, regular, late/at-site)
- Hotel block status (sold out = signal for high demand)

Why hierarchical Bayesian:

- New tournament with 10 entries immediately borrows strength from hundreds of past events
- Naturally produces **credible intervals** (not just point predictions)
- Incorporates domain priors (e.g., "World Open typically draws 400–800")
- Updates cleanly as each new registration arrives (sequential Bayesian updating)
- Handles tournament-specific features (prize fund, fee deadlines) as covariates

Alternative considered: Gaussian Process Regression — good for uncertainty quantification but harder to encode the known S-curve structure.

What NOT to use: Tree-based models (XGBoost, RF) and neural networks cannot extrapolate beyond training data range and need far more data points.

App Architecture

Tech Stack

Layer	Technology	Notes
Backend API	FastAPI (Python)	Async, lightweight, auto-docs
Modeling	PyMC or Stan	Bayesian inference engine
Task Queue	Celery + Redis	Scheduled scraping, report generation
Database	PostgreSQL	Time series storage, tournament metadata
Frontend	React	Dashboard UI
Hosting	AWS / Railway / Fly.io	Start simple, scale later
Notifications	Twilio (SMS) or SendGrid (email)	Anomaly alerts

Data Pipeline

```

[Web Scraper] --every 6hrs--> [Raw Entry Data]
    |
    v
[ETL / Validation] --> [PostgreSQL: entries table]
    |
    v
[Model Refit Service] --on new data--> [Updated predictions + credible intervals]
    |
    v
[Dashboard API] --> [React Frontend]
    |
    v
[Alert Engine] --> [Email/SMS if threshold breached]
    |
    v
[Report Generator] --bi-weekly--> [PDF/Email summary]

```

Database Schema (Core Tables)

```

CREATE TABLE tournaments (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255),
    start_date DATE,
    end_date DATE,
    location VARCHAR(255),
    prize_fund DECIMAL,
    sections JSONB,                -- [{name, fee_tiers: [{deadline, price}]]
    metadata JSONB,               -- competing events, hotel info, confirmed GMs
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE entries (
    id SERIAL PRIMARY KEY,
    tournament_id INT REFERENCES tournaments(id),
    player_name VARCHAR(255),
    section VARCHAR(100),
    registered_at TIMESTAMP,
    scraped_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE predictions (
    id SERIAL PRIMARY KEY,
    tournament_id INT REFERENCES tournaments(id),
    predicted_at TIMESTAMP DEFAULT NOW(),
    model_version VARCHAR(50),

```

```

prediction_date DATE,          -- what date is this prediction FOR
predicted_entries DECIMAL,
ci_lower DECIMAL,             -- 90% credible interval
ci_upper DECIMAL,
parameters JSONB              -- fitted model params snapshot
);

CREATE TABLE alerts (
    id SERIAL PRIMARY KEY,
    tournament_id INT REFERENCES tournaments(id),
    alert_type VARCHAR(50),      -- 'below_threshold', 'above_threshold', 'anomaly'
    threshold_pct DECIMAL,
    actual_value DECIMAL,
    expected_value DECIMAL,
    triggered_at TIMESTAMP DEFAULT NOW(),
    acknowledged BOOLEAN DEFAULT FALSE
);

```

Dashboard Features

Core Views

1. Tournament Overview

- Current entry count vs. predicted count vs. same-point-last-year
- Cumulative entry chart with prediction curve + credible interval band
- Days until event, days until next fee deadline

2. Delta Display (Primary KPI)

- NOT "you have 47 entries" → instead: **"You're tracking 22% below last year at this point in the cycle"**
- Color-coded: green (on track), yellow (watch), red (below threshold)

3. Threshold Alerts

- User-configurable: e.g., "warn me if 10% under last year", "alert if 20% under"
- Push notification / email / SMS when triggered
- Alert history with acknowledge/dismiss

4. What-If Scenario Mode

- "If I extend early bird deadline 2 weeks, what does the model predict?"
- "If I add \$5,000 to the prize fund, what's the expected uplift?"
- Driven by learned covariate effects from historical data

5. Bi-Weekly Report

- Auto-generated for all upcoming tournaments
- Current entries, predicted final, comparison to prior year
- Flagged anomalies
- Delivered via email (PDF attachment or inline HTML)

Tournament Director Onboarding

Critical design principle: Must be dead simple. Tournament directors skew older, not necessarily tech-forward.

Ideal flow:

1. "Paste your tournament registration URL"
2. System auto-detects tournament name, dates, sections from the page
3. Director confirms details, sets alert thresholds
4. Done — scraping begins automatically

Avoid requiring manual data entry wherever possible. Every extra form field costs adoption.

Web Scraper Specification

Target: chessaction.com Entry Lists

```
# Example scraper structure
class CCAEntryScraper:
    """Scrapes Continental Chess Association entry list pages."""

    BASE_URL = "https://onlineregistration.cc/tournaments/advance_entry_list.php"

    def __init__(self, tournament_id: str, session_cookies: dict = None):
        self.tournament_id = tournament_id
        self.session = requests.Session()
```

```

    if session_cookies:
        self.session.cookies.update(session_cookies)

def fetch_entries(self) -> list[dict]:
    """Returns list of {player_name, section, registered_at}."""
    # 1. GET the entry list page
    # 2. Parse HTML table rows
    # 3. Extract player name, section, registration timestamp
    # 4. Deduplicate against existing DB entries
    # 5. Return new entries only
    pass

def detect_changes(self, previous_entries: list) -> dict:
    """Diff current vs previous to find new/removed entries."""
    pass

```

Scraping Schedule

- Every 6 hours for tournaments > 30 days out
- Every 2 hours for tournaments < 30 days out
- Every 30 minutes during event week
- Respect rate limits, add random jitter to timing

Error Handling

- Retry with exponential backoff on failure
- Alert admin if scraper fails 3x consecutively
- Log all scrape results for debugging
- Handle login session expiry gracefully (re-authenticate)

Development Phases

Phase 1: MVP (4–6 weeks)

- Web scraper for chessaction.com entry lists
- PostgreSQL schema + data ingestion pipeline
- Double sigmoid curve fit per tournament (scipy, not Bayesian yet)

- Simple React dashboard: entry chart + prediction curve
- Email alerts when entries cross user-set thresholds
- Single tournament director user

Phase 2: Smart Model (4–6 weeks)

- Ingest 5 years of historical data
- Hierarchical Bayesian model (PyMC)
- Credible intervals on all predictions
- Tournament metadata as covariates (prize fund, fee deadlines, location)
- What-if scenario mode

Phase 3: Multi-Tournament + Reports (3–4 weeks)

- Multi-tournament dashboard view
- Bi-weekly auto-generated reports
- Year-over-year comparison charts
- Multiple user accounts / tournament director login

Phase 4: Growth (Ongoing)

- Support for non-CCA registration platforms
- SMS alerts via Twilio
- Mobile-friendly dashboard
- API for tournament directors to pull predictions programmatically
- Explore partnerships with CCA / USCF for data access

Reference: Current Model Output

From the prototype built on World Open 2026 pre-registration data (20 entries as of March 9, 2026):

$$\text{Model: } f(t) = 20.6 / (1 + e^{(-0.0468 \cdot (t-79.2))}) + 256.7 / (1 + e^{(-0.2512 \cdot (t-246.7))})$$

$$R^2 = 0.964$$

RMSE = 1.09

Key predictions:

Mar 9 (today): ~20 entries
May 15 (early bird): ~21 entries
Jun 28 (late reg): ~53 entries
Jul 2 (event start): ~94 entries
Jul 6 (event end): ~154 entries
Asymptote (L1+L2): ~277 entries

The prototype Excel file (`World_Open_Entry_Prediction.xlsx`) contains three sheets:
Model Summary, Actual Data with residuals, and Prediction Timeline with chart.

Key Risks & Mitigations

Risk	Mitigation
Scraper breaks when site updates HTML	Abstract parser layer; alert on parse failures; have fallback regex patterns
Too few data points early in cycle	Hierarchical model borrows from historical tournaments
Tournament directors don't adopt	Dead-simple onboarding (paste URL, done); demonstrate value with free pilot
Registration platforms block scraping	Pursue data-sharing agreements; offer value back (analytics for their platform)
Model overfits to historical patterns	Cross-validate across tournaments; use informative but not overly tight priors